



รายงานปฏิบัติการที่ 7

การพัฒนาเว็บแอปพลิเคชันด้วย Spring Boot เชื่อมต่อฐานข้อมูล PostgreSQL
และประยุกต์ใช้ Strategy Pattern

จัดทำโดย

ชิตชนก ชนะพา

รหัสนักศึกษา 673380033-9 sec.1

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles Of Software Design

เพื่อประกอบการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

ปีการศึกษา 2569

คำนำ

รายงานปฏิบัติการฉบับนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design โดยมีวัตถุประสงค์เพื่อศึกษาการออกแบบและพัฒนาเว็บแอปพลิเคชันด้วย Spring Boot ร่วมกับ Spring Data JPA และการเชื่อมต่อฐานข้อมูล PostgreSQL ตลอดจนการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์ ได้แก่ GRASP Patterns, SOLID Principles, Layered Architecture และ Strategy Pattern ในการพัฒนาระบบจัดการข้อมูลเกม (Game Catalog Management System)

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านเข้าใจหลักการออกแบบซอฟต์แวร์ การพัฒนาเว็บแอปพลิเคชันด้วย Spring Boot การเชื่อมต่อฐานข้อมูล PostgreSQL และการประยุกต์ใช้ Design Patterns ในการพัฒนาระบบได้อย่างถูกต้อง หากรายงานฉบับนี้มีข้อบกพร่องประการใด ผู้จัดทำขอน้อมรับคำแนะนำและข้อเสนอแนะทุกประการ เพื่อนำไปปรับปรุงและพัฒนาผลงานในโอกาสต่อไป

ชิตชนก ชนะพา

ผู้จัดทำ

สารบัญ

เรื่อง	หน้า
คำนำ	ก
สารบัญ	ข
สารบัญ(ต่อ)	ค
สารบัญรูปภาพ	ง
1. Software Design & Principles Explanation	1
1.1 สถาปัตยกรรมและการประยุกต์ใช้ GRASP Patterns	1
1.1.1 Controller Pattern	1
1.1.2 Information Expert	1
1.1.3 High Cohesion	1
1.1.4 Low Coupling	2
1.1.5 Indirection	2
1.2 High-Level SOLID Principles	2
1.2.1 Single Responsibility Principle (SRP)	2
1.2.2 Open/Closed Principle (OCP)	3
1.2.3 Liskov Substitution Principle (LSP)	3
1.2.4 Interface Segregation Principle (ISP)	3
1.2.5 Dependency Inversion Principle (DIP)	4
1.3 Strategy Pattern	4
1.4 Layered Architecture	5
1.4.1 Controller Layer	5
1.4.2 Service Layer	5
1.4.3 Repository Layer	5
1.4.4 Database Layer	5
1.5 Execution Flow	6

สารบัญ(ต่อ)

เรื่อง	หน้า
2. Code Implementation & Explanation	7
2.1 Entity Layer	7
2.2 Repository Layer	8
2.3 Strategy Package	8
2.4 Service Layer	10
2.5 Controller Layer	11
2.6 Dependency Injection (Constructor Injection)	12
3. Web Application & Database Screenshots	13

สารบัญรูปภาพ

เรื่อง	หน้า
3.1 หน้าจอเพิ่มเกมใหม่ (Create)	13
3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)	13
3.3 หน้าจอแก้ไขเกม (Update)	14
3.4 หน้าจอยืนยันลบ	14
3.5 ผลลัพธ์หลังลบ (Delete)	15
3.6 หน้าจอ Database แสดงข้อมูลจริงในตาราง games	16

1. Software Design & Principles Explanation

1.1 สถาปัตยกรรมและการประยุกต์ใช้ GRASP Patterns

ระบบนี้ถูกออกแบบตามแนวคิด Layered Architecture โดยแบ่งหน้าที่ของแต่ละคลาสออกเป็น Entity, Repository, Service, Controller และ Strategy Package เพื่อให้แต่ละคลาสมีหน้าที่รับผิดชอบที่ชัดเจนและลดการพึ่งพาระหว่างกัน ส่งผลให้ระบบสามารถพัฒนาและบำรุงรักษาได้ง่ายขึ้น

1.1.1 Controller Pattern

GameController ทำหน้าที่เป็นตัวรับ HTTP Request จาก Browser และกำหนดว่าจะเรียกใช้ Service ไດเพื่อประมวลผลข้อมูล โดย Controller ไม่ทำหน้าที่คำนวณหรือจัดการฐานข้อมูลด้วยตนเอง แต่ส่งต่อการทำงานให้ Service Layer ทำให้หน้าที่ของ Controller มีเพียงการรับคำขอและส่งผลลัพธ์กลับไปยัง View

1.1.2 Information Expert

GameService เป็นคลาสที่เหมาะสมที่สุดในการจัดการ Business Logic เนื่องจากเป็นผู้รู้ข้อมูลทั้งหมดเกี่ยวกับการคำนวณราคาหลังหักส่วนลด การเลือก Discount Strategy และการบันทึกข้อมูลลงฐานข้อมูล ดังนั้นการตัดสินใจทางธุรกิจทั้งหมดจึงอยู่ใน Service Layer

1.1.3 High Cohesion

แต่ละคลาสมีหน้าที่เฉพาะเจาะจง เช่น

- Game Entity เก็บข้อมูลเกม
- GameRepository ติดต่อฐานข้อมูล PostgreSQL
- GameService จัดการ Business Logic
- GameController รับ Request จากผู้ใช้
- Strategy Package คำนวณโปรโมชั่น

การแบ่งหน้าที่เช่นนี้ทำให้แต่ละคลาสมีความรับผิดชอบเพียงเรื่องเดียว ส่งผลให้แก้ไขและดูแลรักษาได้ง่าย

1.1.4 Low Coupling

- Controller ไม่ติดต่อฐานข้อมูลโดยตรง แต่เรียกผ่าน Service
- Service ไม่เขียน SQL โดยตรง แต่เรียกผ่าน Repository

การคำนวณส่วนลดไม่ได้เขียนอยู่ใน Service แต่เรียกผ่าน Strategy Pattern การแยกหน้าที่ดังกล่าวช่วยลดการเชื่อมโยงระหว่างคลาสต่าง ๆ ทำให้สามารถเปลี่ยนแปลงส่วนใดส่วนหนึ่งของระบบได้โดยไม่ส่งผลกระทบต่อส่วนอื่น

1.1.5 Indirection

ระบบใช้ Service Layer และ Strategy Pattern เป็นตัวกลางระหว่าง Controller กับ Repository รวมถึงการคำนวณส่วนลด ทำให้แต่ละส่วนของระบบไม่จำเป็นต้องรู้รายละเอียดการทำงานของอีกส่วนหนึ่ง ส่งผลให้ระบบมีความยืดหยุ่นและสามารถขยายเพิ่มเติมได้ง่าย

1.2 High-Level SOLID Principles

1.2.1 Single Responsibility Principle (SRP)

แต่ละคลาสมีหน้าที่เพียงอย่างเดียว

- Game เก็บข้อมูลเกม
- Repository ติดต่อฐานข้อมูล
- Service จัดการ Business Logic
- Controller รับ HTTP Request
- Strategy คำนวณส่วนลด

การแยกหน้าที่เช่นนี้ทำให้แต่ละคลาสสามารถแก้ไขได้โดยไม่กระทบคลาสอื่น

1.2.2 Open/Closed Principle (OCP)

ระบบสามารถเพิ่มโปรโมชั่นใหม่ได้โดยไม่ต้องแก้ไขโค้ดของคลาสเดิม ตัวอย่างเช่น หากต้องการเพิ่ม VIP Discount สามารถสร้าง VIPDiscountStrategy ที่ implements DiscountStrategy แล้วเพิ่มการเลือก Strategy ใน DiscountContext โดยไม่ต้องแก้ไขโค้ดของ Strategy เดิม

1.2.3 Liskov Substitution Principle (LSP)

ทุกคลาสที่ implements DiscountStrategy สามารถใช้แทนกันได้ เช่น

- NoDiscountStrategy
- StudentDiscountStrategy
- SeasonalSaleStrategy

เนื่องจากทุกคลาสมีเมธอดเดียวกันคือ calculateDiscount() จึงสามารถสลับใช้งานได้โดยไม่ทำให้ระบบเกิดข้อผิดพลาด

1.2.4 Interface Segregation Principle (ISP)

DiscountStrategy กำหนดเฉพาะเมธอดที่จำเป็น ได้แก่

- calculateDiscount()
- getDiscountName()

คลาสที่ implements Interface จึงไม่จำเป็นต้องสร้างเมธอดที่ไม่ได้ใช้งาน

1.2.5 Dependency Inversion Principle (DIP)

GameService ไม่สร้าง Discount Strategy ด้วยตนเอง แต่รับ DiscountContext ผ่าน Constructor Injection จาก Spring IoC Container เช่น

```
public GameService(GameRepository gameRepository,
                  DiscountContext discountContext)
```

ทำให้ Service พึ่งพา Abstraction มากกว่าพึ่งพา Implementation โดยตรง

1.3 Strategy Pattern

ระบบใช้ Strategy Pattern สำหรับการคำนวณโปรโมชั่นของเกม

องค์ประกอบของ Pattern ได้แก่

- DiscountStrategy (Interface)
- NoDiscountStrategy
- StudentDiscountStrategy
- SeasonalSaleStrategy
- DiscountContext

DiscountContext จะเลือก Strategy ตามค่า discountType ที่ผู้ใช้เลือก เช่น

- NONE ใช้ NoDiscountStrategy
- STUDENT ใช้ StudentDiscountStrategy
- SEASONAL ใช้ SeasonalSaleStrategy

จากนั้นจะเรียก strategy.calculateDiscount(price) เพื่อคำนวณราคาสุทธิ

ข้อดีของ Strategy Pattern คือสามารถเพิ่มโปรโมชันใหม่ได้ง่าย เช่น

- VIP Discount
- Holiday Discount
- Member Discount

โดยไม่ต้องแก้ไขโค้ดของ Strategy เดิม ซึ่งสอดคล้องกับหลัก Open/Closed Principle

1.4 Layered Architecture

ระบบแบ่งออกเป็น 4 Layer ได้แก่

Browser → Controller → Service → Repository → PostgreSQL

1.4.1 Controller Layer

รับ Request จากผู้ใช้และส่งต่อให้ Service

1.4.2 Service Layer

จัดการ Business Logic เช่น

- เลือก Strategy
- คำนวณราคาหลังหักส่วนลด
- บันทึกข้อมูล

1.4.3 Repository Layer

ติดต่อฐานข้อมูลผ่าน Spring Data JPA

1.4.4 Database Layer

จัดเก็บข้อมูลเกมใน PostgreSQL

การแบ่ง Layer ช่วยให้ระบบมี Low Coupling และ High Cohesion เนื่องจากแต่ละ Layer มีหน้าที่เฉพาะของตนเองและไม่ต้องรู้รายละเอียดการทำงานของ Layer อื่น

1.5 Execution Flow

เมื่อผู้ใช้เปิด Browser และเข้า URL

`http://localhost:8080/games`

- Spring Boot จะส่ง Request ไปยัง GameController
- GameController จะเรียกเมธอด `findAll()` ของ `GameService`
- `GameService` จะเรียก `GameRepository` เพื่อดึงข้อมูลจาก PostgreSQL
- `Repository` จะใช้ Spring Data JPA ติดต่อฐานข้อมูลและคืนรายการเกมกลับมายัง `Service`
- `Service` ส่งข้อมูลกลับไปยัง `Controller`
- `Controller` ส่ง Model ไปยัง Thymeleaf Template (`list.html`)
- Thymeleaf สร้างหน้า HTML และส่งกลับไปยัง Browser เพื่อแสดงผล
- กรณีเพิ่มเกมใหม่ ผู้ใช้กรอกข้อมูลใน `add.html` แล้วกด Save
- `Controller` จะรับข้อมูลผ่านเมธอด `saveGame()`
- จากนั้นเรียก `GameService.save()`
- `GameService` จะเลือก Discount Strategy ตาม `discountType` ผ่าน `DiscountContext`
- `Strategy` จะคำนวณราคาหลังหักส่วนลดและกำหนดชื่อโปรโมชั่น
- `Service` บันทึกข้อมูลผ่าน `GameRepository` ลง PostgreSQL
- เมื่อบันทึกสำเร็จ ระบบจะ Redirect กลับไปยังหน้ารายการเกมและแสดงข้อมูลล่าสุด

2. Code Implementation & Explanation

2.1 Entity Layer

คลาส Game เป็น Entity ที่ใช้เก็บข้อมูลเกมและเชื่อมโยงกับตาราง games ใน PostgreSQL ผ่าน JPA Annotation โดยประกอบด้วยข้อมูล เช่น ชื่อเกม แนวเกม แพลตฟอร์ม คะแนน ราคา วันวางจำหน่าย ประเภทส่วนลด ชื่อโปรโมชั่น และราคาสุทธิหลังหักส่วนลด Entity นี้ทำหน้าที่เป็นแบบจำลองข้อมูล (Data Model) ที่ใช้รับส่งข้อมูลระหว่างฐานข้อมูลกับระบบ

ตัวอย่างโค้ด

```
@Entity
@Table(name = "games")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String genre;
    private String platform;
}
```

Game เป็น Entity ที่ใช้แทนตาราง games ในฐานข้อมูล PostgreSQL โดยใช้ Annotation @Entity และ @Table ของ JPA เพื่อให้ Hibernate สามารถสร้างและจัดการตารางได้อัตโนมัติ คลาสนี้ทำหน้าที่เก็บข้อมูลเกมและเป็นตัวกลางในการรับส่งข้อมูลระหว่างฐานข้อมูลกับระบบ

2.2 Repository Layer

GameRepository สืบทอดจาก JpaRepository<Game, Long> ของ Spring Data JPA ทำให้สามารถใช้งานเมธอดมาตรฐาน เช่น findAll(), findById(), save() และ deleteById() ได้โดยไม่ต้องเขียน SQL เอง ช่วยลดความซับซ้อนของการเข้าถึงฐานข้อมูลและทำให้โค้ดกระชับขึ้น

ตัวอย่างโค้ด

```
@Repository
public interface GameRepository extends JpaRepository<Game, Long> {

}
```

GameRepository สืบทอดจาก JpaRepository ทำให้สามารถใช้งานเมธอดพื้นฐาน เช่น findAll(), findById(), save() และ deleteById() ได้โดยไม่ต้องเขียน SQL เอง ส่งผลให้การเข้าถึงฐานข้อมูลทำได้ง่ายและลดจำนวนโค้ดลง

2.3 Strategy Package

Strategy Package ประกอบด้วย DiscountStrategy ซึ่งเป็น Interface และคลาส NoDiscountStrategy, StudentDiscountStrategy และ SeasonalSaleStrategy ที่เป็นผู้คำนวณส่วนลดแต่ละประเภท ส่วน DiscountContext ทำหน้าที่เลือก Strategy ที่เหมาะสมตามค่า discountType ของเกม แล้วส่งกลับไปให้ Service ใช้งาน การออกแบบลักษณะนี้ช่วยให้สามารถเพิ่มรูปแบบส่วนลดใหม่ได้โดยไม่ต้องแก้ไขโค้ดเดิม

ตัวอย่างโค้ด

```
public interface DiscountStrategy {

    double calculateDiscount(double price);

    String getDiscountName();
}

@Component
public class StudentDiscountStrategy implements DiscountStrategy {

    @Override
    public double calculateDiscount(double price) {
        return price * 0.90;
    }

    @Override
    public String getDiscountName() {
        return "Student Discount";
    }
}
```

Strategy Pattern ใช้แยกวิธีการคำนวณส่วนลดออกจาก Service โดย DiscountStrategy เป็น Interface ส่วน StudentDiscountStrategy, SeasonalSaleStrategy และ NoDiscountStrategy เป็น Implementation ของแต่ละโปรโมชัน ทำให้สามารถเพิ่มรูปแบบส่วนลดใหม่ได้โดยไม่กระทบโค้ดเดิม

2.4 Service Layer

GameService เป็นศูนย์กลางของ Business Logic โดยทำหน้าที่รับข้อมูลจาก Controller เลือก Discount Strategy ผ่าน DiscountContext คำนวณราคาสุทธิและชื่อโปรโมชั่น จากนั้นจึงบันทึกข้อมูลผ่าน GameRepository นอกจากนี้ยังจัดการการค้นหา การแก้ไข และการลบข้อมูลเกม ทำให้ Controller ไม่ต้องมี Business Logic อยู่ภายใน

ตัวอย่างโค้ด

```
@Service
public class GameService {

    private final GameRepository gameRepository;
    private final DiscountContext discountContext;

    public GameService(GameRepository gameRepository,
                       DiscountContext discountContext) {
        this.gameRepository = gameRepository;
        this.discountContext = discountContext;
    }
}
```

GameService ใช้ Constructor Injection เพื่อรับ GameRepository และ DiscountContext จาก Spring IoC Container แทนการสร้างออบเจกต์ด้วยคำสั่ง new ภายในคลาส วิธีนี้ช่วยลดการพึ่งพาระหว่างคลาส ทำให้สามารถทดสอบ (Unit Testing) และบำรุงรักษาได้ง่ายขึ้น อีกทั้งยังสอดคล้องกับหลัก Dependency Inversion Principle (DIP) ของ SOLID

2.5 Controller Layer

GameController รับ HTTP Request จาก Browser เช่น การแสดงรายการเกม การเพิ่ม การแก้ไข และการลบข้อมูล จากนั้นเรียกใช้เมธอดที่เหมาะสมใน GameService และส่งผลลัพธ์ไปยัง Thymeleaf Template (list.html, add.html, edit.html, delete.html) เพื่อสร้างหน้าเว็บตอบกลับผู้ใช้

ตัวอย่างโค้ด

```
@Controller
@RequestMapping("/games")
public class GameController {

    private final GameService gameService;

    public GameController(GameService gameService) {
        this.gameService = gameService;
    }
}
```

GameController รับ GameService ผ่าน Constructor Injection โดย Spring จะสร้างและส่งออบเจกต์ GameService ให้โดยอัตโนมัติ Controller จึงมีหน้าที่เพียงรับ HTTP Request เรียกใช้ Service และส่งข้อมูลไปยัง View โดยไม่ต้องทราบรายละเอียดการทำงานภายในของ Service

2.6 Dependency Injection (Constructor Injection)

ระบบใช้ Constructor Injection ในทุก Layer เพื่อให้ Spring IoC Container เป็นผู้สร้างและจัดการ Dependency เช่น GameController รับ GameService ผ่าน Constructor และ GameService รับ GameRepository กับ DiscountContext ผ่าน Constructor เช่นเดียวกัน วิธีนี้ช่วยลดการสร้างออบเจกต์ด้วยตนเอง ทำให้โค้ดมีความยืดหยุ่น ทดสอบได้ง่าย และสอดคล้องกับหลัก Dependency Inversion Principle (DIP) ของ SOLID รวมถึงเป็นแนวทางที่ Spring Framework แนะนำสำหรับการจัดการ Dependency ในแอปพลิเคชันขนาดใหญ่

ตัวอย่างโค้ด

```
public GameController(GameService gameService) {
    this.gameService = gameService;
}

public GameService(GameRepository gameRepository,
    DiscountContext discountContext) {
    this.gameRepository = gameRepository;
    this.discountContext = discountContext;
}
```

โปรเจกต์นี้ใช้ Constructor Injection ในทุก Layer ที่มี Dependency โดย Spring IoC Container จะเป็นผู้สร้างและจัดการออบเจกต์ทั้งหมด ทำให้แต่ละคลาสไม่ต้องสร้าง Dependency เอง ส่งผลให้ระบบมี Low Coupling, สนับสนุนการทดสอบได้ง่าย และเป็นแนวทางที่ Spring Framework แนะนำในการพัฒนาแอปพลิเคชัน

3. Web Application & Database Screenshots

Game Catalog CP353002 — Lab 7

← กลับไปรายการเกม

➕ เพิ่มเกมใหม่
กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม
Silent Hill f (673380033-9 SEC 1)

แนวเกม (Genre)
Horror

แพลตฟอร์ม (Platform)
PC

คะแนน (Rating)
7

ราคาปกติ (บาท)
2190

% ส่วนลด (Strategy)
ราคาปกติ (0%)

วันวางจำหน่าย (Release Date)
09/25/2025

บันทึก ยกเลิก

3.1 หน้าจอเพิ่มเกมใหม่ (Create)

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

➕ เพิ่มเกมใหม่

เพิ่มเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Minecraft	Open World	PC	★ 10.0	23/06/2026	ราคาปกติ	1500.00	
2	The sims	Life Simulation	PC	★ 8.5	19/07/2026	ส่วนลดนักศึกษา (10%)	2250.00 2500.00	
3	peak	Action	PC	★ 7.5	26/08/2026	ส่วนลดเทศกาล (20%)	280.00 350.00	
4	Silent Hill f (673380033-9 SEC 1)	Horror	PC	★ 7.0	25/09/2025	ราคาปกติ	2190.00	

ทั้งหมด 4 เกม

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)

← กลับไปรายการเกม

แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Silent Hill f (673380033-9 SEC 1)

ชื่อเกม
Silent Hill f (673380033-9 SEC 1)

แนวเกม (Genre)
Horror

แพลตฟอร์ม (Platform)
PC

คะแนน (Rating)
7.0

ราคาปกติ (บาท)
2190.0

% ส่วนลด (Strategy)
ราคาปกติ (0%)

วันวางจำหน่าย (Release Date)
mm/dd/yyyy

3.3 หน้าจอแก้ไขเกม (Update)

← กลับไปรายการเกม

ยืนยันการลบ

คุณแน่ใจหรือไม่?
การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม	peak
แนวเกม	Action
แพลตฟอร์ม	PC
คะแนน	★ 7.5
ราคา	350.00 บาท

3.4 หน้าจอยืนยันลบ

← → ↻ 📑 localhost:8080/games ☆ 📌 School 📖 All Bookmarks

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete [+ เพิ่มเกมใหม่](#)

🟢 ลงเกมสำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชัน (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Minecraft	Open World	PC	★ 10.0	23/06/2026	ราคาปกติ	1500.00	✎ 🗑
2	The sims	Life Simulation	PC	★ 8.5	19/07/2026	ส่วนลดนักศึกษา (10%)	2250.00 2500.00	✎ 🗑
3	Silent Hill f (673380033-9 SEC 1)	Horror	PC	★ 7.0	25/09/2025	ราคาปกติ	2190.00	✎ 🗑

📄 ทั้งหมด 3 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3.5 ผลลัพธ์หลังลบ (Delete)

pgAdmin 4

File Object Tools Edit View Window Help

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × public.games/lab7/postgres@PostgreSQL 18 ×

public.games/lab7/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM public.games
2 ORDER BY id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

id [PK] bigint	discount_type character varying (255)	genre character varying (255)	platform character varying (255)	price double precision	rating double precision	release_date date	title character varying (255)	discount_name character varying (255)
1	2 NONE	Open World	PC	1500	10	2026-06-23	Minecraft	ราคาปกติ
2	3 STUDENT	Life Simulation	PC	2500	8.5	2026-07-19	The sims	ส่วนลดนักศึกษา (10%)
3	6 NONE	Horror	PC	2190	7	2025-09-25	Silent Hill f (673380033-9 SEC...	ราคาปกติ

Total rows: 3 Query complete 00:00:00.145 CRLF Ln 2, Col 17

pgAdmin 4

File Object Tools Edit View Window Help

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × public.games/lab7/postgres@PostgreSQL 18 ×

public.games/lab7/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM public.games
2 ORDER BY id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

id [PK] bigint	discount_type character varying (255)	genre character varying (255)	platform character varying (255)	price double precision	rating double precision	release_date date	title character varying (255)	discount_name character varying (255)	final_price double precision
1	2 NONE	Open World	PC	1500	10	2026-06-23	Minecraft	ราคาปกติ	1500
2	3 STUDENT	Life Simulation	PC	2500	8.5	2026-07-19	The sims	ส่วนลดนักศึกษา (10%)	2250
3	6 NONE	Horror	PC	2190	7	2025-09-25	Silent Hill f (673380033-9 SEC...	ราคาปกติ	2190

Total rows: 3 Query complete 00:00:00.145 CRLF Ln 2, Col 17

3.6 หน้าจอ Database แสดงข้อมูลจริงในตาราง games